

#Ejercicio Nro: 8

##Enunciado

Práctica 1

Ejercicios sobre Metodologías de Desarrollo en Cascada

1. Análisis de Requerimientos:

Ejercicio 1: Un cliente te solicita una aplicación web para gestionar su inventario. Define los requisitos funcionales y no funcionales del sistema.

Ejercicio 2: Redacta un caso de uso para la funcionalidad de "Agregar un nuevo producto" en la aplicación web del ejercicio 1.

2. Diseño del Sistema:

Ejercicio 3: Elabora un diagrama de flujo de datos para la aplicación web del ejercicio 1.

Ejercicio 4: Diseña la interfaz de usuario para la pantalla de "Inicio" de la aplicación web del ejercicio 1.

3. Diseño del Programa:

Ejercicio 5: Elige una arquitectura adecuada para la aplicación web del ejercicio 1 y justifica tu elección.

Ejercicio 6: Diseña la base de datos para la aplicación web del ejercicio 1.

4. Diseño:

Utilizando los siguientes diagrama resuelva los casos de usos de los ejercicios 7 y 8:

Diagrama de Dominio: Identifica las entidades, atributos y relaciones del sistema.

Diagrama de Robustez: Analiza cómo el sistema responde a diferentes escenarios de uso.

Prototipo: Crea una versión simplificada del sistema para probar la usabilidad y funcionalidad.

Diagrama de Secuencia: Describe la interacción entre los diferentes objetos del sistema.

Diagrama de Clases: Define las clases, sus atributos, métodos y relaciones

Ejercicio 7: Implementa la funcionalidad de "Agregar un nuevo producto" en la aplicación web del ejercicio 1 utilizando el lenguaje de programación de tu preferencia.

Ejercicio 8: Implementa la lógica de negocio para la funcionalidad de "Agregar un nuevo producto" en la aplicación web del ejercicio 1.

5. Pruebas:

Ejercicio 9: Define un conjunto de pruebas unitarias para la funcionalidad de "Agregar un nuevo producto" en la aplicación web del ejercicio 1.

Ejercicio 10: Ejecuta pruebas de integración para la funcionalidad de "Agregar un nuevo producto" en la aplicación web del ejercicio 1.

6. Despliegue del Programa:

Ejercicio 11: Definir un plan de despliegue para la aplicación web del ejercicio 1.

Ejercicio 12: Despliega la aplicación web del ejercicio 1 en un servidor de producción.

7. Mantenimiento:

Ejercicio 13: Definir un plan de mantenimiento para la aplicación web del ejercicio 1.

Ejercicio 14: Implementa una corrección de errores para un problema detectado en la aplicación web del ejercicio 1.

8. Nos preparamos para nuevos retos

Ejercicio 15: Arme un equipo de trabajo y defina los roles para realizar los ejercicios anteriores para un futuro dominio de aplicación relacionado con inteligencia artificial generativa.

##Resolución

Integrante: Teo Fassardi

Parte 1 Análisis de Requerimientos

Ejercicio 1:

Requisitos funcionales:

- 1- Registrar productos nuevos en el inventario, indicando datos como nombre, código, categoría, precio, cantidad disponible y descripción.
- 2- Modificar la información de los productos existentes, por ejemplo, cambiar el precio, actualizar la cantidad disponible o corregir datos cargados previamente.
- 3- Eliminar productos del inventario cuando ya no se vendan o no formen parte del stock de la empresa.
- 4- Consultar el listado de productos disponibles, mostrando información como nombre, código, categoría, precio y stock actual.
- 5- Buscar productos mediante filtros, como nombre, código, categoría o disponibilidad.
- 6- Controlar el stock de cada producto, permitiendo saber cuántas unidades hay disponibles.
- 7- Registrar entradas de mercadería, es decir, aumentar el stock cuando ingresan nuevos productos al inventario.
- 8- Registrar salidas de mercadería, descontando unidades cuando se realiza una venta, entrega o retiro de productos.
- 9- Generar alertas de bajo stock cuando la cantidad disponible de un producto sea menor al mínimo establecido.
- 10- Permitir el acceso de usuarios autorizados, mediante inicio de sesión con usuario y contraseña.
- 11- Generar reportes básicos del inventario, como productos con bajo stock, productos disponibles y movimientos de entrada y salida.

Requisitos no funcionales:

- 1- Seguridad: el sistema debe proteger la información del inventario y permitir el acceso solo a usuarios autorizados.
- 2- Usabilidad: la aplicación debe ser clara, sencilla e intuitiva, para que los usuarios puedan utilizarla sin dificultad.

- 3- Rendimiento: el sistema debe responder de manera rápida al consultar, cargar, modificar o eliminar productos.
- 4- Disponibilidad: la aplicación debe estar disponible la mayor parte del tiempo para que el cliente pueda gestionar el inventario cuando lo necesite.
- 5- Escalabilidad: el sistema debe permitir agregar más productos, usuarios o funcionalidades en el futuro sin afectar su funcionamiento.
- 6- Compatibilidad: la aplicación debe poder utilizarse desde distintos navegadores web, como Chrome, Edge o Firefox.
- 7- Mantenibilidad: el sistema debe estar desarrollado de manera ordenada para facilitar futuras correcciones, mejoras o actualizaciones.
- 8- Integridad de datos: la información cargada debe mantenerse correcta y coherente, evitando errores como cantidades negativas o productos duplicados.

Ejercicio 2:

Caso de uso: Agregar un nuevo producto

Nombre del caso de uso:

Agregar un nuevo producto.

Actor principal:

Usuario encargado del inventario.

Objetivo:

Permitir que el usuario registre un nuevo producto dentro del sistema de inventario.

Descripción:

El usuario ingresa al sistema, accede a la sección de productos y selecciona la opción para agregar un nuevo producto. Luego completa los datos solicitados por el sistema, como nombre, código, categoría, precio, cantidad disponible y descripción. Finalmente, el sistema valida la información ingresada y guarda el nuevo producto en el inventario.

Precondiciones:

El usuario debe haber iniciado sesión en el sistema.

El usuario debe tener permisos para administrar productos.

Flujo principal:

1. El usuario ingresa al sistema con su usuario y contraseña.
2. El sistema muestra el menú principal.

3. El usuario selecciona la opción “Productos”.
4. El sistema muestra el listado de productos existentes.
5. El usuario selecciona la opción “Agregar nuevo producto”.
6. El sistema muestra un formulario de carga.
7. El usuario completa los datos del producto: nombre, código, categoría, precio, stock inicial y descripción.
8. El usuario confirma la carga del producto.
9. El sistema valida que los datos ingresados sean correctos.
10. El sistema guarda el nuevo producto en el inventario.
11. El sistema muestra un mensaje de confirmación indicando que el producto fue agregado correctamente.

Flujos alternativos:

A1: Datos incompletos

Si el usuario deja campos obligatorios vacíos, el sistema muestra un mensaje de error indicando que debe completar todos los datos requeridos.

A2: Código de producto duplicado

Si el código ingresado ya pertenece a otro producto, el sistema informa que el producto ya existe y solicita ingresar un código diferente.

A3: Datos inválidos

Si el usuario ingresa un precio o una cantidad de stock con valores negativos, el sistema muestra un mensaje de error y solicita corregir los datos.

Postcondiciones:

El producto queda registrado en el sistema.

El inventario se actualiza con el nuevo producto cargado.

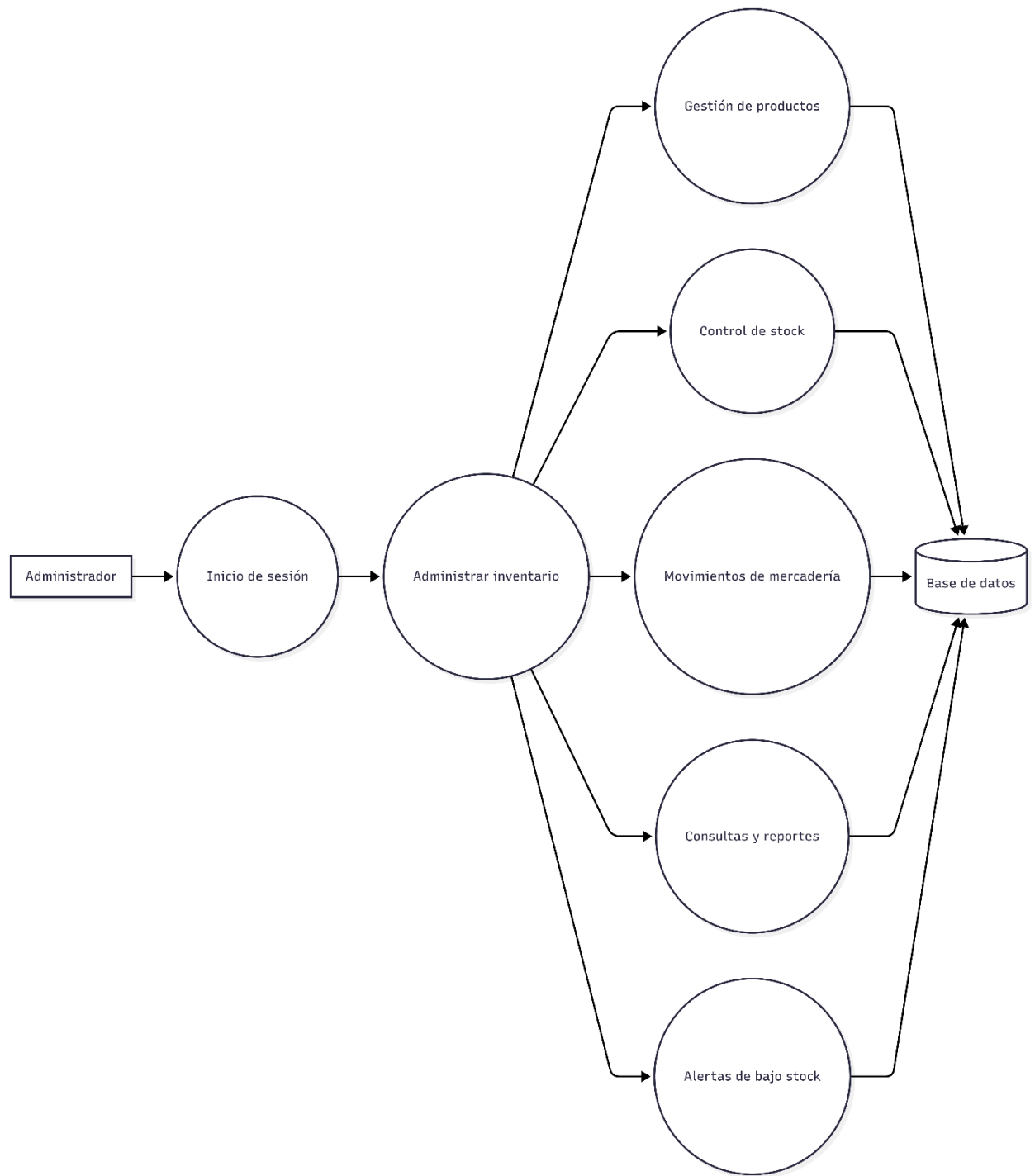
El usuario puede consultar, modificar o eliminar el producto posteriormente.

Resultado esperado:

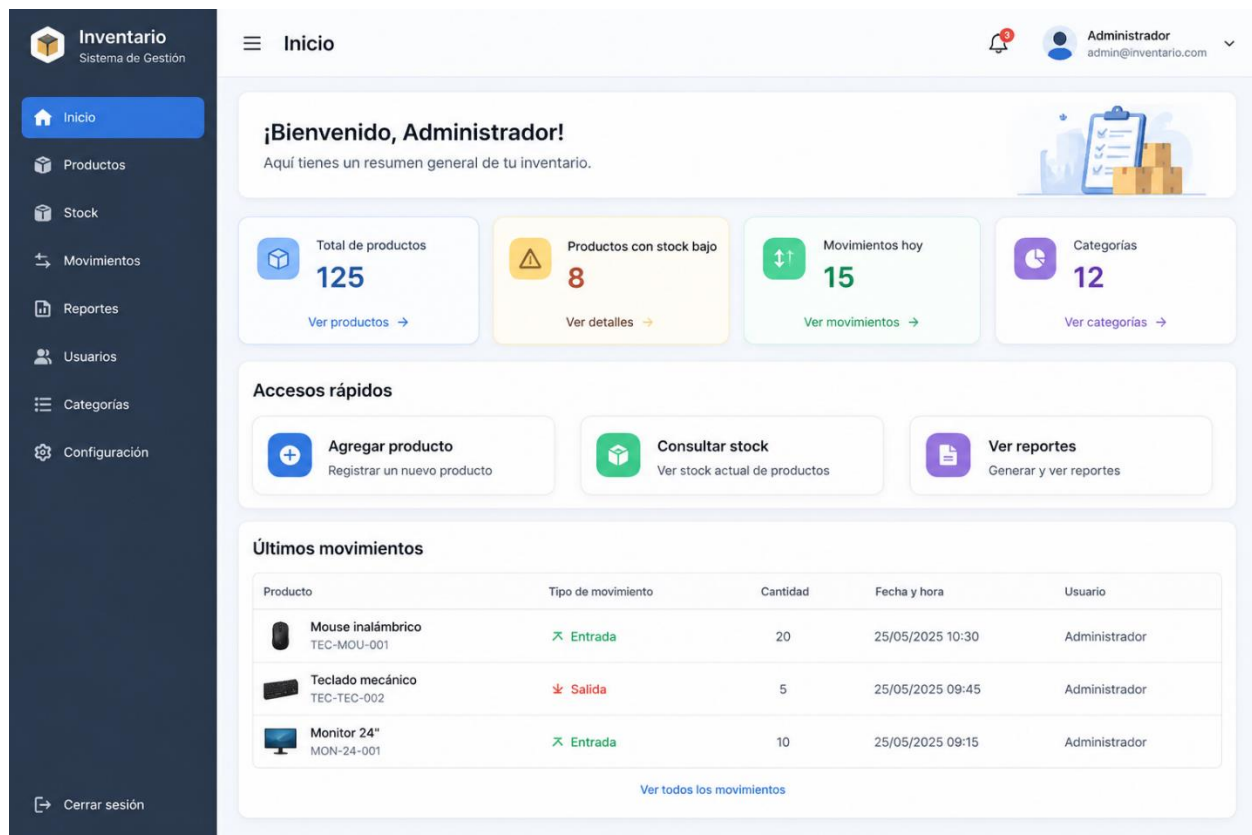
El sistema permite agregar correctamente un nuevo producto al inventario, siempre que los datos ingresados sean válidos y no exista otro producto con el mismo código.

Parte 2 Diseño del Sistema

Ejercicio 3:



Ejercicio 4:



Parte 3 Diseño del Programa

Ejercicio 5:

La arquitectura elegida para la aplicación web de inventario será una arquitectura cliente-servidor:

Para el Front-end: se utilizará Next.js con React, ya que permite desarrollar una interfaz web dinámica, organizada y fácil de usar para el administrador del inventario.

Para el Back-end: se utilizarán las funciones del servidor de Next.js, que permitirán crear las operaciones necesarias del sistema, como agregar productos, modificar datos, consultar stock, eliminar productos y generar reportes.

Para la base de datos: se utilizará SQL Server, administrado mediante SQL Server Management Studio, donde se guardará la información de productos, categorías, usuarios y movimientos de stock.

La elección de esta arquitectura se debe a que permite separar la interfaz del usuario, la lógica del sistema y el almacenamiento de datos.

Ejercicio 6:

```
create table usuarios(  
id_usuario int primary key,  
nombre nvarchar(60),  
email nvarchar(100),  
contraseña nvarchar(50),  
rol nvarchar(30)  
);
```

```
create table categorias(  
id_categoria int primary key,  
nombre nvarchar(60),  
descripcion nvarchar(100)  
);
```

```
create table productos(  
id_producto int primary key,  
nombre nvarchar(60),  
codigo nvarchar(30),  
descripcion nvarchar(100),  
precio decimal(10,2),  
stock_actual int,  
stock_minimo int,  
id_categoria int foreign key references categorias(id_categoria)  
);
```

```
create table movimientos_stock(  
id_movimiento int primary key,  
tipo_movimiento nvarchar(20),  
cantidad int,  
fecha date,  
observacion nvarchar(100),  
id_producto int foreign key references productos(id_producto),  
id_usuario int foreign key references usuarios(id_usuario)  
);
```

Parte 4 Diseño

Ejercicio 7:

Diagrama de dominio:

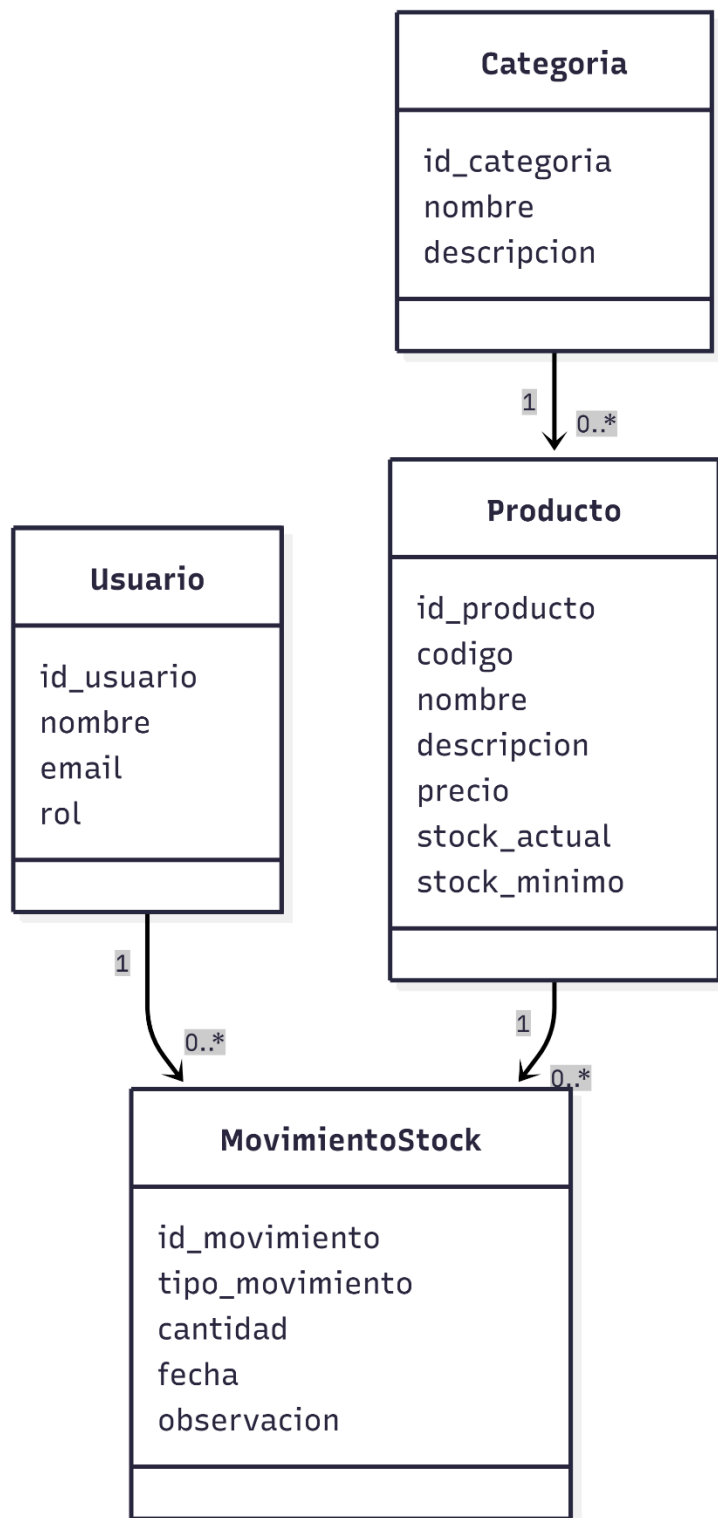


Diagrama de Robustez:

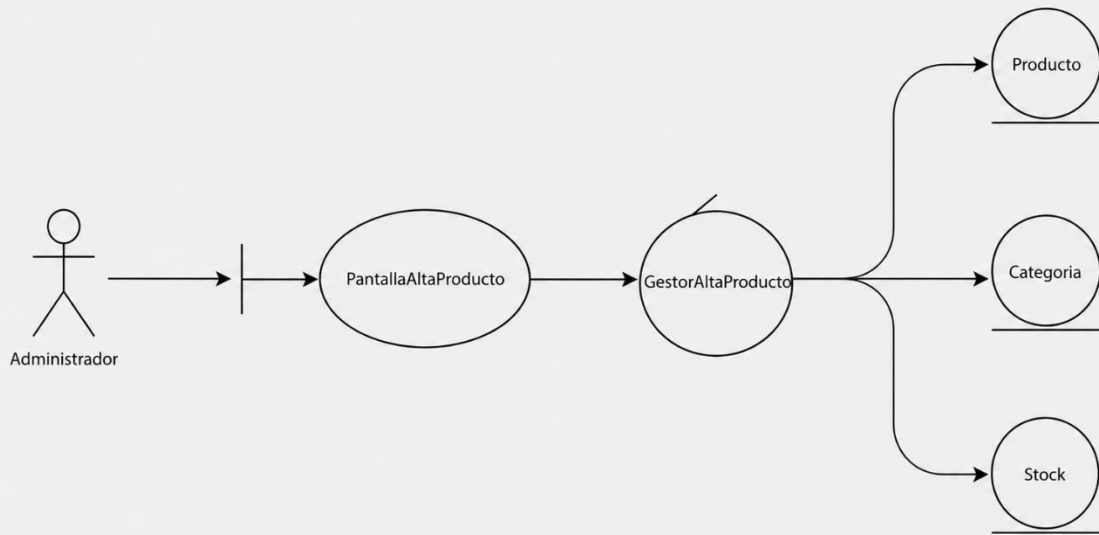


Diagrama de Secuencia:

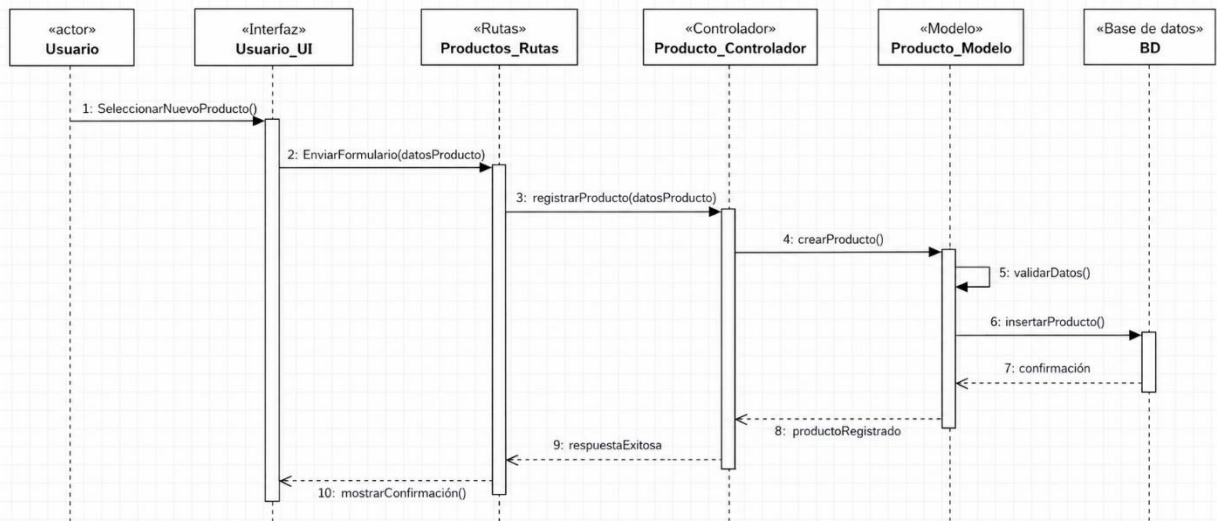
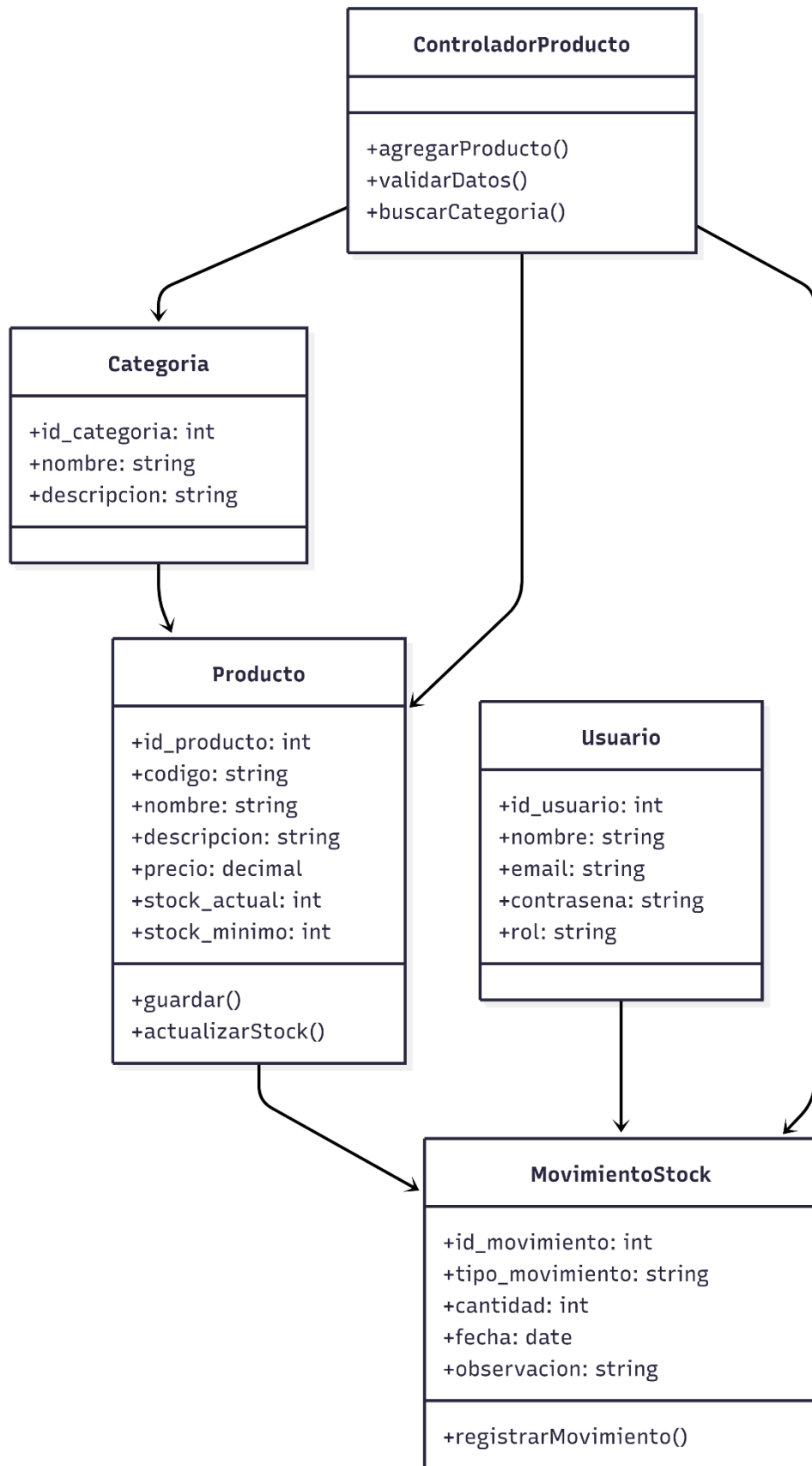


Diagrama de clases:



Prototipo:

 Sistema de Inventario

 Administrador ▾

 Inicio

 Productos

 Stock

 Movimientos

 Reportes

 Ayuda

Alta de producto

Complete los datos para registrar un nuevo producto

Código

Ingrese el código

Nombre

Ingrese el nombre del producto

Categoría

Seleccione una categoría ▾

Precio

\$ Ingrese el precio

Stock inicial

Ingrese el stock inicial

Stock mínimo

Ingrese el stock mínimo

Descripción

Ingrese una descripción del producto

Cancelar

Guardar

Código:

```
1  "use client";
2
3  import { useState } from "react";
4
5  export default function AltaProducto() {
6    const [codigo, setCodigo] = useState("");
7    const [nombre, setNombre] = useState("");
8    const [categoria, setCategoria] = useState("");
9    const [precio, setPrecio] = useState("");
10   const [stockInicial, setStockInicial] = useState("");
11   const [stockMinimo, setStockMinimo] = useState("");
12   const [descripcion, setDescripcion] = useState("");
13
14   function limpiarFormulario() {
15     setCodigo("");
16     setNombre("");
17     setCategoria("");
18     setPrecio("");
19     setStockInicial("");
20     setStockMinimo("");
21     setDescripcion("");
22   }
23
24   function guardarProducto(e: React.FormEvent) {
25     e.preventDefault();
26
27     if (
28       codigo === "" ||
29       nombre === "" ||
30       categoria === "" ||
31       precio === "" ||
32       stockInicial === "" ||
33       stockMinimo === ""
```

```
5   export default function AltaProducto() {
24     function guardarProducto(e: React.FormEvent) {
26
27       if (
28         codigo === "" ||
29         nombre === "" ||
30         categoria === "" ||
31         precio === "" ||
32         stockInicial === "" ||
33         stockMinimo === ""
34       ) {
35         alert("Complete todos los campos obligatorios.");
36         return;
37       }
38
39       if (Number(precio) <= 0) {
40         alert("El precio debe ser mayor a cero.");
41         return;
42       }
43
44       if (Number(stockInicial) < 0 || Number(stockMinimo) < 0) {
45         alert("El stock no puede ser negativo.");
46         return;
47       }
48
49       alert("Producto agregado correctamente.");
50       limpiarFormulario();
51     }
52   }
```

```
52
53   return (
54     <form onSubmit={guardarProducto}>
55       <h2>Alta de producto</h2>
56       <p>Complete los datos para registrar un nuevo producto</p>
57
58       <input
59         type="text"
60         placeholder="Código"
61         value={codigo}
62         onChange={(e) => setCodigo(e.target.value)}
63       />
64
65       <input
66         type="text"
67         placeholder="Nombre"
68         value={nombre}
69         onChange={(e) => setNombre(e.target.value)}
70       />
71
72       <select
73         value={categoria}
74         onChange={(e) => setCategoria(e.target.value)}
75       >
76         <option value="">Seleccione una categoría</option>
77         <option value="Tecnología">Tecnología</option>
78         <option value="Librería">Librería</option>
79         <option value="Herramientas">Herramientas</option>
80       </select>
```

```
82     <input
83       type="number"
84       placeholder="Precio"
85       value={precio}
86       onChange={(e) => setPrecio(e.target.value)}
87     />
88
89     <input
90       type="number"
91       placeholder="Stock inicial"
92       value={stockInicial}
93       onChange={(e) => setStockInicial(e.target.value)}
94     />
95
96     <input
97       type="number"
98       placeholder="Stock mínimo"
99       value={stockMinimo}
100      onChange={(e) => setStockMinimo(e.target.value)}
101    />
102
103    <textarea
104      placeholder="Descripción"
105      value={descripcion}
106      onChange={(e) => setDescription(e.target.value)}
107    />
108
109    <button type="button" onClick={limpiarFormulario}>
110      Cancelar
111    </button>
```



```

112
113     <button type="submit">
114         Guardar
115     </button>
116 </form>
117 );
118 }

```

Ejercicio 8:

Cuando el usuario completa el formulario de alta de producto y presiona el botón Guardar, el sistema recibe los datos ingresados y realiza una serie de validaciones.

Primero, verifica que los campos obligatorios estén completos. El producto debe tener código, nombre, precio, stock inicial, stock mínimo y una categoría seleccionada.

Luego, el sistema controla que el precio sea mayor a cero, ya que no tendría sentido registrar un producto con precio negativo o igual a cero. También verifica que el stock inicial y el stock mínimo no sean valores negativos.

Después, el sistema comprueba que el código del producto no esté repetido. Para eso, consulta la base de datos y verifica si ya existe otro producto con el mismo código. Si el código ya existe, el sistema no permite guardar el producto y muestra un mensaje de error.

También se valida que la categoría seleccionada exista en la base de datos. Si la categoría no existe, el producto no se registra.

Si todas las validaciones son correctas, el sistema guarda el nuevo producto en la tabla productos. Además, si el producto se carga con stock inicial mayor a cero, se registra un movimiento de stock de tipo Entrada en la tabla movimientos_stock.

Finalmente, el sistema muestra un mensaje de confirmación indicando que el producto fue agregado correctamente.

Parte 5 Pruebas

Ejercicio 9:

Para comprobar que la función “Agregar un nuevo producto” funciona correctamente, se pueden definir pruebas unitarias sobre los casos más importantes del registro. En primer lugar, se debe probar la carga de un producto con datos válidos, por ejemplo: nombre, descripción, categoría, precio y stock inicial. Si todos los campos están completos y los valores son correctos, el sistema debería guardar el producto en el inventario, mostrar un mensaje de confirmación y permitir que aparezca en el listado de productos.

También se debe probar qué ocurre cuando faltan datos obligatorios o cuando se ingresan valores incorrectos. Por ejemplo, si el nombre del producto está vacío, el sistema debería rechazar el registro e indicar que ese campo es obligatorio. Lo mismo debería suceder si el precio o el stock inicial tienen valores negativos, ya que no corresponde guardar un producto con un precio menor a cero o con una cantidad de stock inválida. En esos casos, el sistema no debería guardar el producto en la base de datos y tendría que mostrar un mensaje de error claro.

Otra prueba útil consiste en intentar agregar un producto que ya existe en el inventario. En ese caso, el sistema debería detectar la repetición y evitar que se genere un registro duplicado. Como resultado esperado, podría mostrar un aviso indicando que el producto ya fue cargado o permitir actualizar la información existente, según la lógica definida para la aplicación. Estas pruebas permiten validar que la función sea confiable y que el inventario mantenga datos correctos.

Ejercicio 10:

Pruebas de integración para “Agregar un nuevo producto”

Carga correcta desde la interfaz: verificar que el usuario pueda completar el formulario de alta de producto y enviar los datos al sistema.

Comunicación entre interfaz y ruta: comprobar que, al presionar el botón Guardar, los datos ingresados sean enviados correctamente desde Usuario_UI hacia Productos_Rutas.

Comunicación entre ruta y controlador: verificar que Productos_Rutas reciba la solicitud y la envíe correctamente a Producto_Controlador.

Validación desde el controlador: comprobar que el controlador valide los datos antes de enviarlos al modelo.

Registro en la base de datos: verificar que, si los datos son correctos, el producto quede guardado correctamente en la BD.

Relación con categoría: comprobar que el producto se registre asociado a una categoría existente.

Respuesta al usuario: verificar que, luego de guardar el producto, el sistema muestre un mensaje de confirmación.

Manejo de errores: comprobar que, si ocurre un error, como código repetido o categoría inválida, el sistema muestre un mensaje claro y no guarde el producto.

Parte 6 Despliegue del Programa

Ejercicio 11:

Plan de despliegue:

Para realizar el despliegue de la aplicación web de gestión de inventario, primero se debe preparar el entorno donde va a funcionar el sistema. En esta etapa se revisan los requisitos necesarios, como el servidor, la base de datos, los archivos del proyecto y la conexión entre la aplicación y el almacenamiento de información. También se deben definir los usuarios responsables del despliegue y verificar que el sistema esté listo para pasar desde una versión de prueba a una versión operativa.

Luego se debe configurar el entorno de producción. Esto incluye subir los archivos de la aplicación, organizar las carpetas del proyecto, cargar la base de datos inicial y revisar que las tablas necesarias estén creadas correctamente. También se debe comprobar que las funciones principales, como inicio de sesión, carga de productos, registro de movimientos, control de stock y generación de reportes, funcionen sin errores. Antes de habilitar el sistema para su uso, conviene hacer una prueba completa con datos simulados para confirmar que la aplicación responde correctamente.

Finalmente, una vez realizadas las pruebas, se publica la aplicación para que pueda ser utilizada por el administrador. El despliegue debería hacerse en un momento de baja actividad para evitar inconvenientes. Después de la publicación, se controla que el sistema permita ingresar, registrar productos, actualizar el stock y consultar reportes. Si aparece algún error, se corrige antes de dar por finalizado el proceso. Cuando todo funciona correctamente, se considera que la aplicación quedó desplegada y lista para su uso.

Ejercicio 12:

Despliegue de la aplicación web:

Para desplegar la aplicación web de gestión de inventario en un servidor de producción, primero se debe revisar que el proyecto esté completo y funcionando correctamente en el entorno de prueba. Esto incluye verificar los archivos principales de la aplicación, como el HTML, los estilos CSS y el código JavaScript, además de comprobar que las funciones de inicio de sesión, carga de productos, movimientos de mercadería, control de stock y reportes respondan de forma adecuada.

Luego se debe subir la aplicación al servidor elegido y configurar la base de datos que almacenará la información del inventario. En esta etapa se cargan las tablas necesarias, se revisa la conexión entre la aplicación y la base de datos, y se verifica que los datos puedan guardarse y consultarse sin errores. También se recomienda configurar medidas básicas de seguridad, como usuarios con permisos adecuados, acceso protegido y copias de respaldo.

Una vez publicada la aplicación, se realizan pruebas finales desde el servidor de producción. El objetivo es comprobar que el administrador pueda ingresar al sistema, agregar productos, registrar entradas y salidas, consultar stock y visualizar reportes. Si todo funciona correctamente, la aplicación queda disponible para su uso. En caso de detectar errores, se corrigen antes de considerar terminado el despliegue.

Parte 7 Mantenimiento

Ejercicio 13:

El mantenimiento de la aplicación web de gestión de inventario debe incluir revisiones periódicas para asegurar que el sistema continúe funcionando de manera correcta. Para esto, se deben controlar el acceso al sistema, el registro de productos, los movimientos de mercadería, las consultas de stock y la generación de reportes. También conviene revisar posibles errores informados por los usuarios y corregirlos para evitar problemas en el uso diario.

Otro aspecto importante del mantenimiento es la actualización del sistema. Esto implica mejorar el código cuando sea necesario, corregir fallas, optimizar la carga de la aplicación y revisar que la base de datos conserve información correcta. Además, si el inventario crece o aparecen nuevas necesidades, se pueden agregar funciones como filtros de búsqueda, reportes más detallados o alertas de bajo stock más completas.

Por último, el plan debe contemplar copias de seguridad de la información. Como el sistema trabaja con productos, cantidades y movimientos, es necesario proteger esos datos ante posibles errores técnicos o pérdidas de información. Realizar respaldos periódicos permite recuperar la base de datos si ocurre un problema y ayuda a mantener la continuidad del uso de la aplicación.

Ejercicio 14:

Implementación de una corrección de errores:

Se detectó un error en la funcionalidad Agregar producto: el sistema permitía guardar productos con stock inicial negativo.

Para corregir este problema, se agregó una validación antes de guardar el producto en la base de datos.

Problema detectado:

El usuario podía ingresar un valor negativo en el campo stock_actual.

Corrección aplicada:

Antes de registrar el producto, el sistema verifica que el stock inicial sea mayor o igual a cero.

Si el stock inicial es menor a 0:

mostrar mensaje de error

no guardar el producto

Resultado esperado:

El sistema ya no permite registrar productos con stock negativo. Si el usuario intenta hacerlo, se muestra un mensaje de error indicando que el stock inicial no puede ser menor a cero.

Parte 8 Nos preparamos para nuevos retos

Ejercicio 15:

Para desarrollar una aplicación web que utilice inteligencia artificial generativa, se propone formar un equipo de trabajo con perfiles técnicos y funcionales. El proyecto puede estar orientado a una aplicación que ayude a generar respuestas automáticas, textos, recomendaciones o consultas inteligentes dentro del sistema. Para comenzar, se necesita un responsable del proyecto que organice las tareas, controle los avances y mantenga la comunicación entre los integrantes del equipo. También se requiere un analista funcional, encargado de relevar las necesidades del sistema, definir qué debe hacer la aplicación y transformar esos requerimientos en funciones concretas.

Dentro del equipo técnico, se necesita un desarrollador frontend para construir la interfaz que utilizará el usuario y un desarrollador backend para programar la lógica del sistema, la conexión con la base de datos y la integración con el modelo de inteligencia artificial. Además, puede participar un especialista en datos o inteligencia artificial, encargado de revisar cómo se enviarán las consultas al modelo, cómo se procesarán las respuestas y qué controles se aplicarán para que la información generada sea útil y coherente con el objetivo de la aplicación.

Por último, se debe incluir un perfil de testing o control de calidad, que pruebe el funcionamiento completo del sistema antes de entregarlo. Esta persona verifica que la aplicación responda correctamente, que no existan errores en los formularios, que la información se guarde bien y que las respuestas generadas por la inteligencia artificial sean adecuadas. Con este equipo, el proyecto queda organizado por roles y cada integrante participa en una parte específica del desarrollo, desde el análisis inicial hasta la prueba y puesta en funcionamiento.